


UNIVERSITÀ DI PARMA
Dipartimento di Ingegneria e Architettura

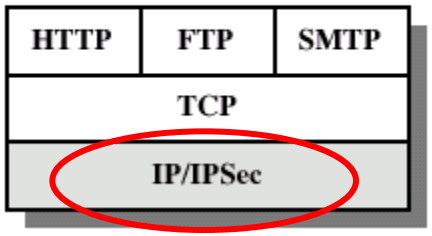
# Secure Communications: IPSec and TLS

Luca Veltri

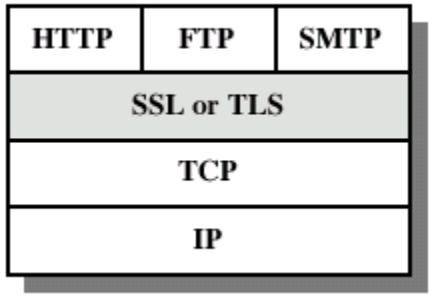
(mail.to: luca.veltri@unipr.it)

Course of Cybersecurity, 2025/2026

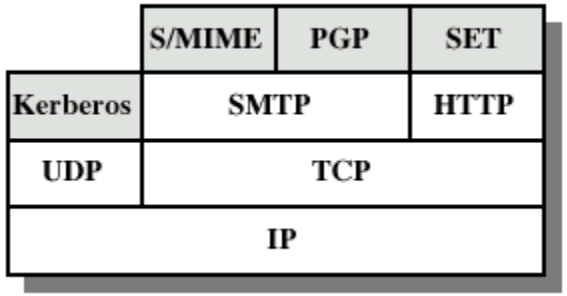
# Security within IP stack protocols




Network level




transport/session level




Application level

# IPSec

# IPSec Introduction

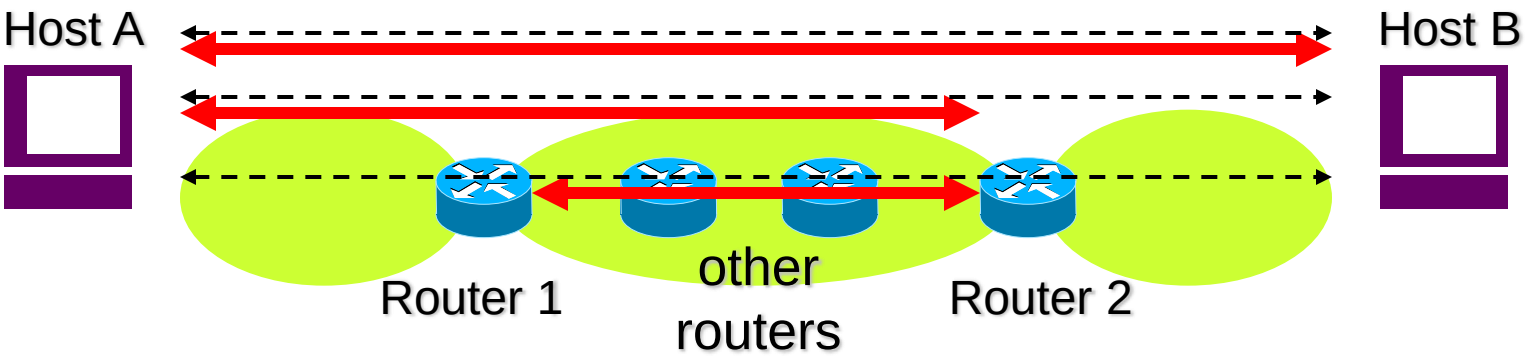
- IPSec, as defined in RFC 4301 and successive standards, is a framework for providing interoperable, high quality, cryptographically-based security for IPv4 and IPv6
  - **offering protection in a standard fashion for all protocols that may be carried over IP (including IP itself)**
- The set of security services offered includes
  - **access control**
  - **connectionless data integrity, data origin authentication**
  - **detection and rejection of replays**
  - **confidentiality**
  - **and limited traffic flow confidentiality**

# IPSec Introduction (cont.)

- An IPSec implementation operates in a host, or as a security gateway (SG), affording protection to IP traffic
  - **a security gateway is an intermediate system implementing IPSec, e.g., a firewall or router that has been IPSec-enabled**
- IPSec does not adversely affect users, hosts, and other Internet components that do not employ IPSec for traffic protection
  - **transparent for intermediate nodes that do not implement it**
  - **transparent for end nodes (host) that do not implement it, when implemented by intermediate nodes**
  - **transparent for end applications that are not aware of it**
- Hence, IPSec is very useful in presence of legacy hosts or applications
- One IPSec implementation may protect traffic from/to several hosts and applications

# IPSec Introduction (cont.)

- IPsec can be used to protect one or more "paths"
  - **between a pair of hosts (host-to-host)**
    - end-to-end, alternative to transport or application level security solutions
  - **between a pair of security gateways (router-to-router)**
    - often referred to as VPN/IPSec
  - **between a security gateway and a host (host-to-router)**
    - also referred to as VPN/IPSec or “road-warrior”



# Transport & Tunnel Modes

- Original packet:

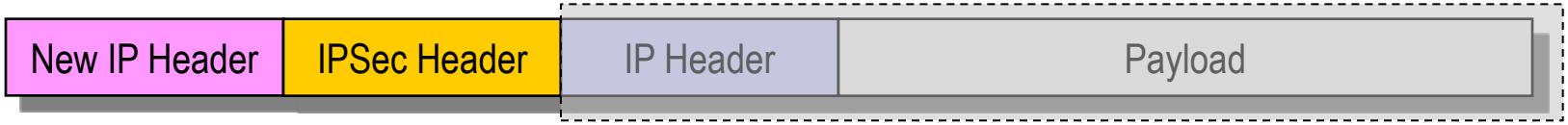



- IPSec can be used according to two modes:
  - **transport mode**
    - IPSec provides protection to upper layer protocols
    - the source and destination must be the endpoints




original payload encrypted/authenticated

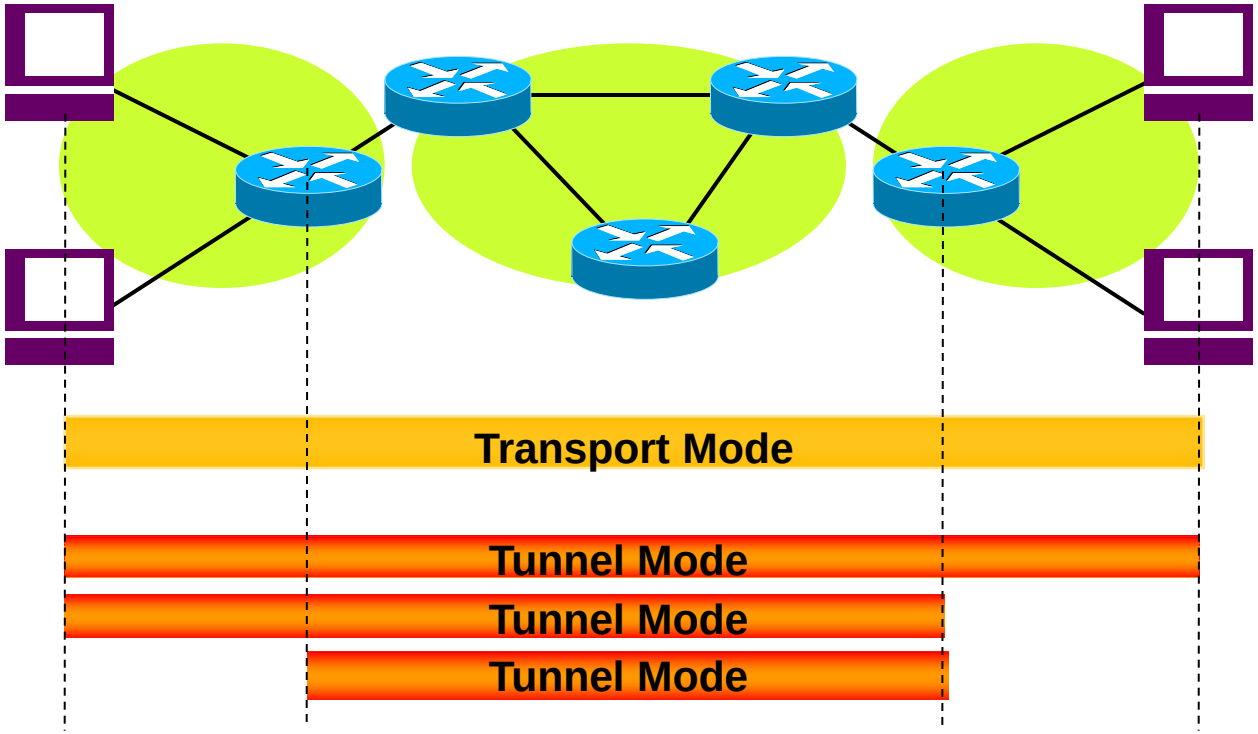
  - **tunnel mode**
    - IPSec is applied to tunneled IP packets
    - the outer IP header Source and Destination Addresses identify the "endpoints" of the IPSec tunnel
    - the inner IP header Source and Destination Addresses identify the original sender and recipient of the datagram (the actual endpoints)




original packet encrypted/authentcated

# Transport & Tunnel Modes (cont.)

- Transport mode can be used only when source and destination addresses coincide with the addresses of the IPSec endpoints



# IPSec protocols

- Two protocols have been originally specified for securing IPSec communications
  - **the Authentication Header (AH)**
    - originally designed to offer integrity and data origin authentication (i.e. message authenticity), with optional (at the discretion of the receiver) anti-replay protection
    - no confidentiality
    - rarely used
  - **Encapsulating Security Payload (ESP)**
    - offers integrity, data origin authentication, data confidentiality, and anti-replay protection
    - there are also provisions for limited traffic flow confidentiality
      - i.e., provisions for concealing packet length, and for facilitating efficient generation and discard of dummy packets

# IPSec protocols (cont.)

- Support for AH is optional and experience has shown that there are very few contexts in which ESP cannot provide the requisite security services
  - **in fact ESP can provide confidentiality, authenticity, or both,**
  - **as a result, AH is rarely used**

- Because IPSec security services require the use of cryptographic keys, IPSec relies on a separate mechanism for putting these keys in place
  - **IPSec specifies IKE (Internet Key Exchange) as public-key based key management mechanism**
  - **other automated key distribution techniques may also be used**

# Cryptographic algorithm independency

- IPsec security protocols (ESP, AH, and IKE) are designed to be cryptographic algorithm independent
  - **this modularity permits selection of different sets of cryptographic algorithms as appropriate, without affecting the other parts of the implementation**
  - **different users may select different sets of cryptographic algorithms**
  - **to facilitate interoperability in the global Internet, a set of default cryptographic algorithms for use with AH and ESP, and a set of mandatory-to-implement algorithms for IKEv2, are specified**

# Security Association (SA)

- A Security Association (SA) is a simplex logical "connection" that affords security services to the traffic carried by it
  - **specifies algorithms, key material, and parameters to operate the AH and/or ESP operations**
  - **a major function of IKE is the establishment and maintenance of SAs**
  - **to secure typical, bi-directional communication between two IPsec-enabled systems, a pair of SAs (one in each direction) is required**



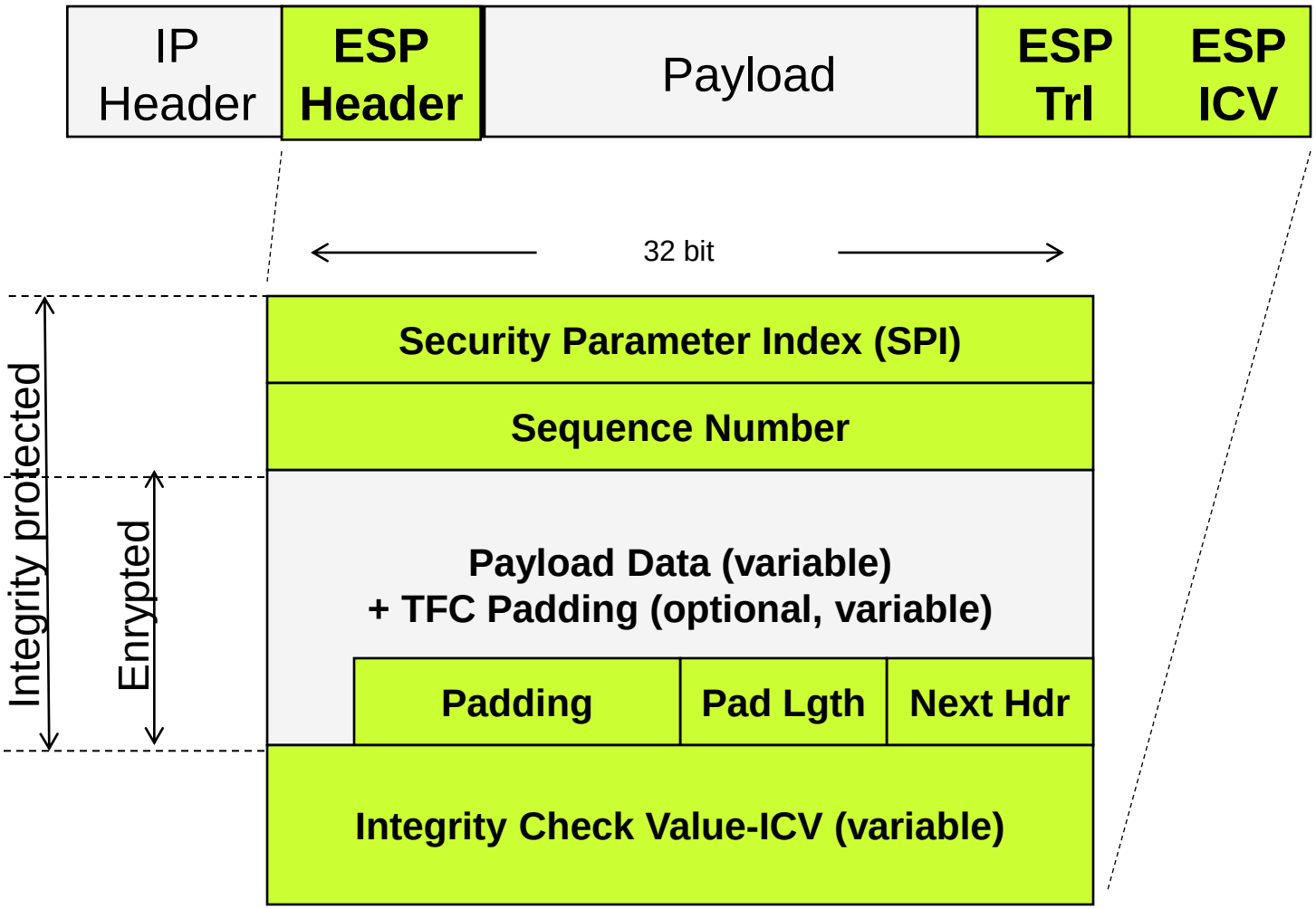
# Encapsulating Security Payload (ESP)

- Encapsulating Security Payload (ESP) (RFC 4303) is designed to provide different security services
  - **confidentiality**
  - **data origin authentication and message integrity**
  - **anti-replay service (a form of partial sequence integrity)**
  - **(limited) traffic flow confidentiality**
    - by adding padding bytes after the end of the payload data
- Confidentiality and integrity can be offered independently
  - **only data authentication**
    - ESP is simpler and faster to process than AH
  - **only encryption**
    - this will provide defense only against passive attackers
      - using encryption without a strong integrity mechanism may render the confidentiality service insecure against some forms of active attacks
  - **both services**
    - typical case

# ESP (cont.)

- The anti-replay service may be used only if the integrity service is provided
  - **the selection of this service is solely at the discretion of the receiver and thus need not be negotiated**
- The traffic flow confidentiality (TFC) service generally is effective only if
  - **ESP is employed in a fashion that conceals the ultimate source and destination addresses of correspondents, e.g., in tunnel mode between SEGs, and**
  - **sufficient traffic flows between IPsec peers (either naturally or as a result of generation of masking traffic) to conceal the characteristics of specific, individual subscriber traffic flows**

# ESP Packet Format



# ESP Packet Format (cont.)

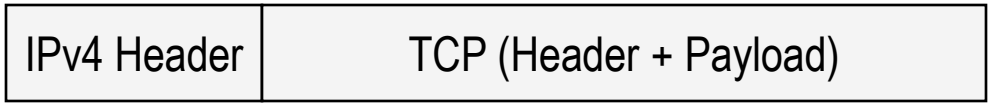
- Security Parameter Index (SPI)
  - **an arbitrary 32-bit value that is used by a receiver to identify the SA to which an incoming packet is bound**
- Sequence number
  - **unsigned 32-bit field contains a counter value that increases by one for each packet sent, i.e., a per-SA packet sequence number**
- Payload data
  - **variable-length field containing data (from the original IP packet) described by the Next Header field**
    - If the algorithm used to encrypt the payload requires cryptographic synchronization data, e.g., an Initialization Vector (IV), then this data is carried explicitly in the Payload field
- Traffic Flow Confidentiality (TFC) Padding
  - **added only if the Payload Data field contains a specification of the length of the IP datagram**
    - this is always true in tunnel mode, and may be true in transport mode depending on whether the next layer protocol (e.g., IP, UDP, ICMP) contains explicit length information

# ESP Packet Format (cont.)

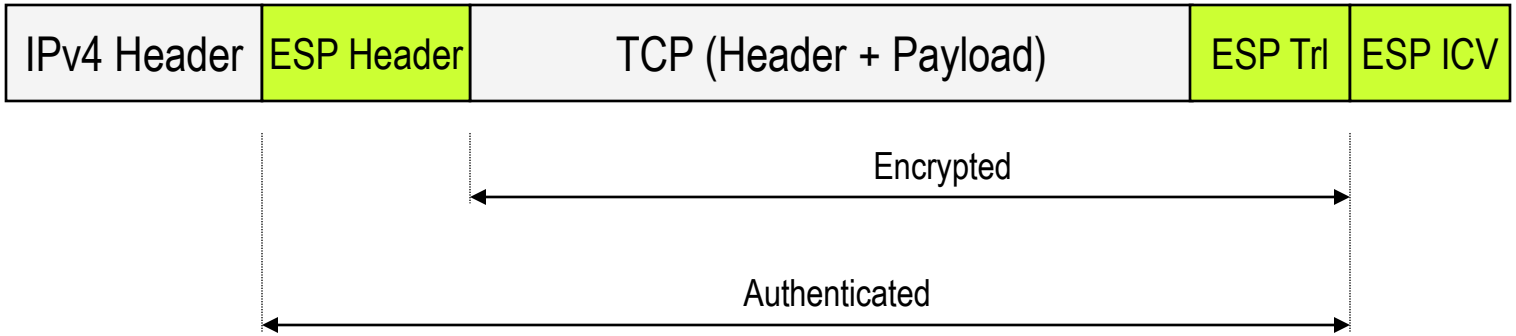
- Padding (for Encryption)
  - **0 to 255 bytes of padding**
    - If the encryption algorithm requires the plaintext to be a multiple of some number of bytes, e.g., the block size of a block cipher, or to ensure that the resulting ciphertext terminates on a 4-byte boundary
    - a separate mechanism should be used for TFC (if required)
- Pad length
  - **the number of pad bytes in the Padding field**
- Next header
  - **8-bit field that identifies the type of data contained in the Payload Data field, e.g., an IPv4 or IPv6 packet, or a next layer header and data**
- Integrity Check Value (ICV)
  - **(optional) variable-length field computed over the ESP header, Payload, and ESP trailer fields**
  - **implicit ESP trailer fields (integrity padding and high-order Extended Sequence Numbers bits, if applicable) are included in the ICV computation**
- If the confidentiality service is selected, the encrypted data consists of the Payload Data and the ESP trailer
- If the integrity service is selected, the integrity computation encompasses the SPI, Sequence Number, Payload Data, and the ESP trailer

# ESP - Transport Mode

- In transport mode, ESP is inserted after the IP header and before a next layer protocol, e.g., TCP, UDP, ICMP, etc.
  - **in IPv4, this translates to placing ESP after the IP header (and any options that it contains), but before the next layer protocol**
- Example of IPv4 packet before applying ESP

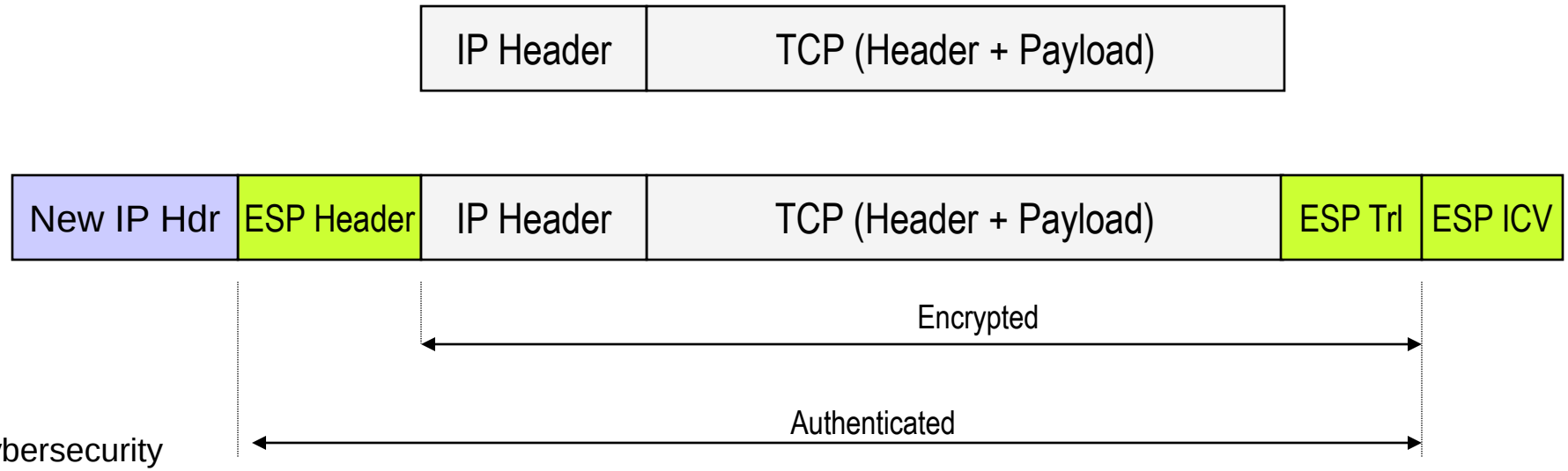



- The same packet after applying ESP in transport mode



# ESP - Tunnel Mode

- In tunnel mode, the "inner" IP header carries the ultimate (IP) source and destination addresses, while an "outer" IP header contains the addresses of the IPsec "peers", e.g., addresses of SEGs
  - **mixed inner and outer IP versions are allowed, i.e., IPv6 over IPv4 and IPv4 over IPv6**
  - **ESP protects the entire inner IP packet, including the entire inner IP header**



# Internet Key Exchange (IKEv2)

- IP Security (IPSec) provides confidentiality, data integrity, access control, and data source authentication to IP datagrams
- These services are provided by maintaining shared state (the Security Association) between the source and the destination
  - **this state defines the specific services provided to the datagram, which cryptographic algorithms will be used to provide the services, and the keys used as input to the cryptographic algorithms, etc.**
  - **establishing this state in a manual fashion does not scale well**
- Internet Key Exchange protocol (IKEv2)
  - **performs mutual authentication between two parties**
  - **establishes an IKE SA (IKE_SA) that includes shared secret information that can be used to efficiently establish SA for ESP (CHILD_SA)**

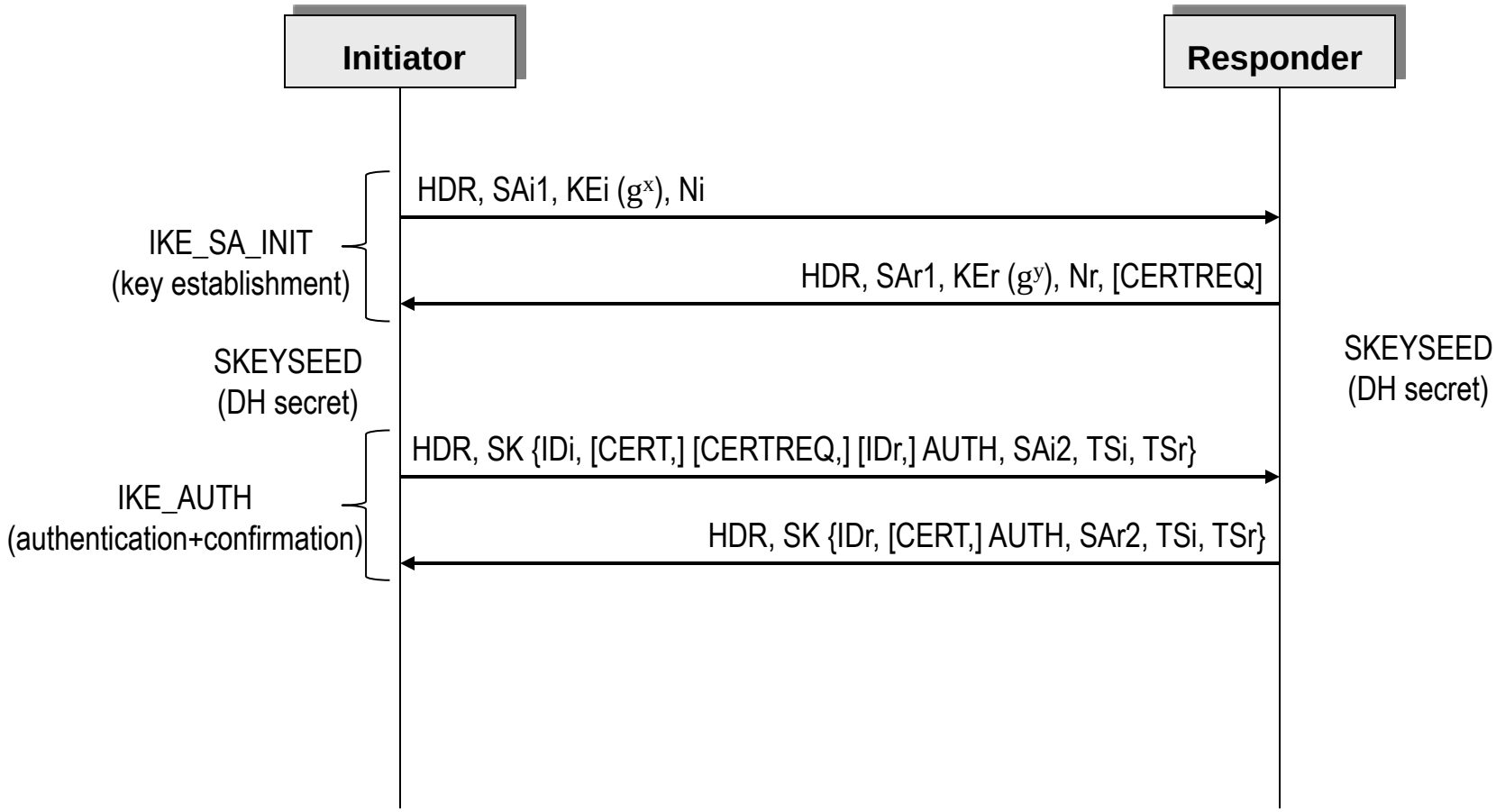
# IKEv2 (cont.)

- IKEv2 implements a Authenticated Diffie–Hellman (SIGMA-type design) key exchange
  - **provides full AKE with mutual (or unilateral) explicit key authentication, identity agreement, and forward secrecy**
  - **key agreement + forward secrecy via**
    - Ephemeral Diffie–Hellman (or ECDH)
  - **authentication via**
    - digital signatures (certificates), or
    - PSK (pre-shared key), or
    - EAP (for client auth)

# IKE Exchanges

- All IKE communications consist of pairs of messages (called exchange)
  - **a request and a response (with retransmission)**
- The first exchange is IKE_SA_INIT
  - **negotiates security parameters for the IKE_SA, sends nonces, and sends Diffie-Hellman values**
- The second exchange is IKE_AUTH
  - **transmits identities, proves knowledge of the secrets corresponding to the two identities, and sets up an SA for the first ESP or AH CHILD_SA**
  - **parts of these messages are encrypted and integrity protected with keys established through the IKE_SA_INIT exchange**
- Subsequent exchanges are CREATE_CHILD_SA
  - **creates other CHILD_SA**
  - **MAY be initiated by either ends of the IKE_SA**

# IKE_SA_INIT and IKE_AUTH exchanges



# IKE_SA_INIT and IKE_AUTH exchanges (cont.)

- Notation:

| | |
|---|---|
| AUTH | **Authentication** |
| CERT | **Certificate** |
| CERTREQ | **Certificate Request** |
| HDR | **IKE header (not a payload)** |
| IDi | **Identification - Initiator** |
| IDr | **Identification - Responder** |
| KE | **Key Exchange** |
| Ni, Nr | **Nonce** |
| SA | **Security Association** |
| SK | **Encrypted and Authenticated** |
| TSi | **Traffic Selector - Initiator** |
| TSr | **Traffic Selector - Responder** |

# IKE_SA_INIT and IKE_AUTH exchanges (cont.)

- Description:
  - HDR **contains the SPIs, version numbers, and flags**
  - SAi **and** SAr **state (offer/choice) the cryptographic algorithms for the IKE_SA**
  - **The** KE **contains the Diffie-Hellman value**
  - Ni **and** Nr **are nonce values**
  - **the initiator and responder assert their identities with the** ID **payload**
  - AUTH **is a signature or MAC over the contents of the IKE_SA_INIT messages**
    - the actual algorithm depends on the type of keys
      - in case of shared secret:
        - AUTH = prf(prf(SharedSecret,"Key Pad for IKEv2"), <data>)
          - the inner PRF with a pad string is used so that if the shared secret is derived from a password, the IKE implementation can store the value prf(Shared Secret,"Key Pad for IKEv2") instead of the clear password
      - in case of public-key
        - RSA or DSA digital signature over the hash value

# IKE_SA_INIT and IKE_AUTH exchanges (cont.)

- Description (cont.):
  - **optionally, messages 3 and 4 may include a certificate, or certificate chain providing evidence that the key used to compute a digital signature belongs to the name in the** ID **payload**
  - **the optional payloads** TSi **and** TSr **report the proposed traffic selectors (3rd message) and the accepted subset of traffic selectors (4th message)**

# IKE_SA_INIT and IKE_AUTH exchanges (cont.)

- Description (cont.):
  - SKEYSEED **is the DH secret from which all keys are derived for that IKE_SA**
    - SKEYSEED is calculated as: $= \mathrm{PRF}(\mathrm{Ni} \,||\, \mathrm{Nr}, \mathrm{g\hat{}ir})$
      - where PRF(,) is negotiated within the SA, e.g. HMAC
    - from SKEYSEED further secrets are then generated:

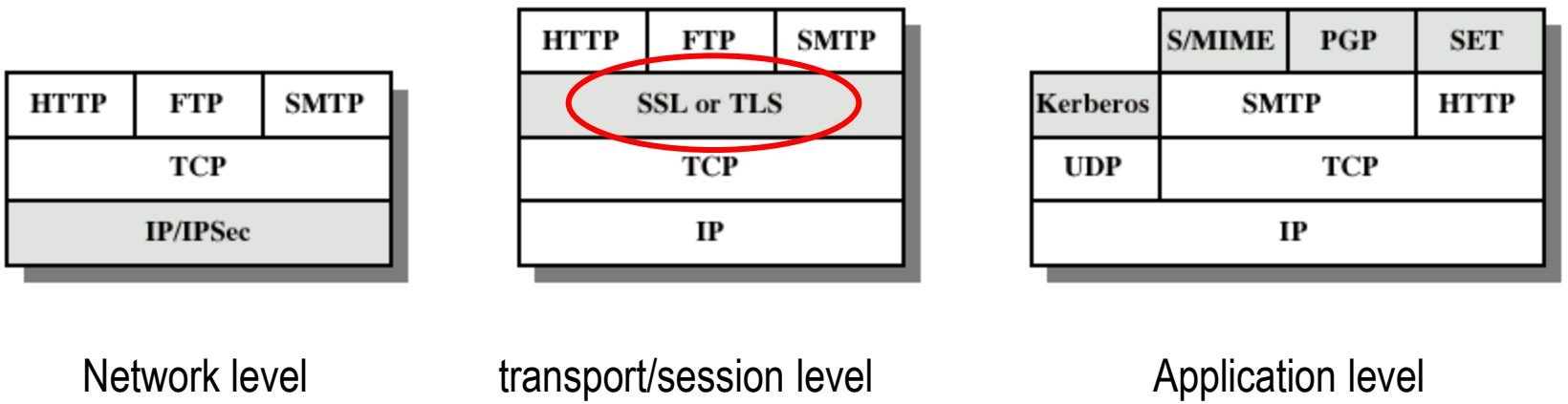
$$\mathrm{SK_d} \,||\, \mathrm{SK_ai} \,||\, \mathrm{SK_ar} \,||\, \mathrm{SK_ei} \,||\, \mathrm{SK_er} \,||\, \mathrm{SK_pi} \,||\, \mathrm{SK_pr} = \\ = \mathrm{prf+}(\mathrm{SKEYSEED}, \mathrm{Ni} \,||\, \mathrm{Nr} \,||\, \mathrm{SPIi} \,||\, \mathrm{SPIr})$$

      - where
        - prf+(,) is a PRF built on prf(,)
        - SK_e and SK_a are used for IKE encryption and integrity protection
        - SK_d is used for derivation of further keying material for CHILD_SAs
        - SK_p are used in the input data of AUTH
  - **all but the headers of all the messages that follow the first exchange are encrypted and integrity protected by** SK{.}

# TLS and DTLS

# Transport Layer Security (TLS)

- Transport Layer Security (TLS) protocol (RFC 8446, version 1.3) provides privacy and data integrity between two communicating client/server applications
- TLS is application protocol independent
  - **higher-level protocols can layer on top of the TLS protocol transparently**



# TLS security services

- TLS provides:
  - **peer's identity authentication**
    - using asymmetric cryptography (e.g., RSA, DSA, etc.)
  - **negotiation of encryption algorithm and cryptographic keys**
    - protected against eavesdropping, modifying and replay attacks
  - **confidentiality**
    - symmetric cryptography is used
  - **Message integrity and data authentication**
    - message transport includes a message integrity check using a keyed MAC
    - secure hash functions (e.g., SHA-1, etc.) are used for MAC computations
- TLS uses X.509 server certificates and client certificates (optional)

# TLS vs. SSL

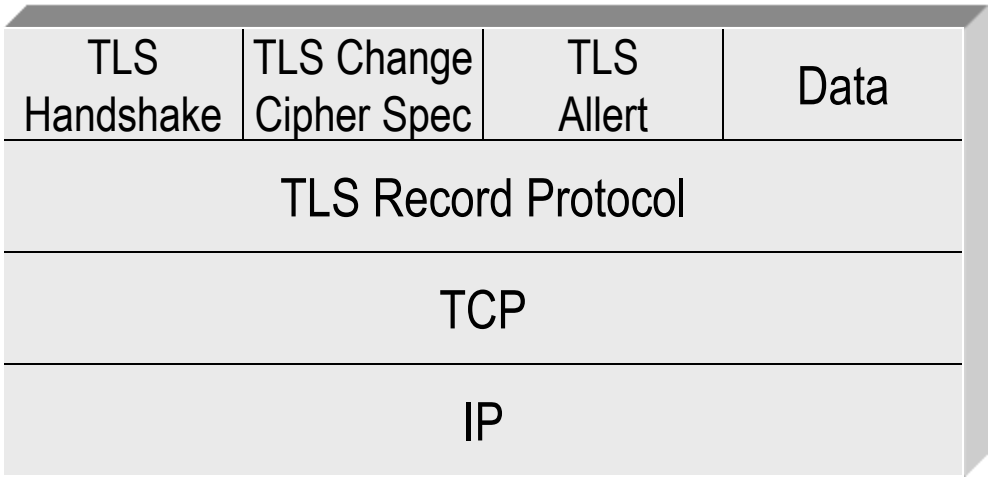
- TLS protocol (IETF standard) was based on the SSL 3.0 Protocol Specification as published by Netscape
- TLS v1.0 == SSL v3.1
  - **the differences between TLS v1.0 and SSL 3.0 were minor:**
    - record format version number
    - uses HMAC for MAC
    - a pseudo-random function expands secrets
    - has additional alert codes
    - some changes in supported ciphers
    - changes in certificate negotiations
    - changes in use of padding

# TLS phases

- 1) Handshake
  - **Establish connection**
  - **Agree on encryption algorithm**
  - **Exchange key**
  - **Authentication**
    - through certificates
    - server only authentication, or both client and server
- 2) Securing messages
  - **Sending the actual enchrypted messages**
  - **Integrity checks with MACs**

# TLS protocol

- The TLS Protocol is a layered protocol
  - **At each layer, messages may include fields for length, description, and content**
- The protocol is composed of two layers:
  - **TLS Handshake Protocol**
  - **TLS Record Protocol**

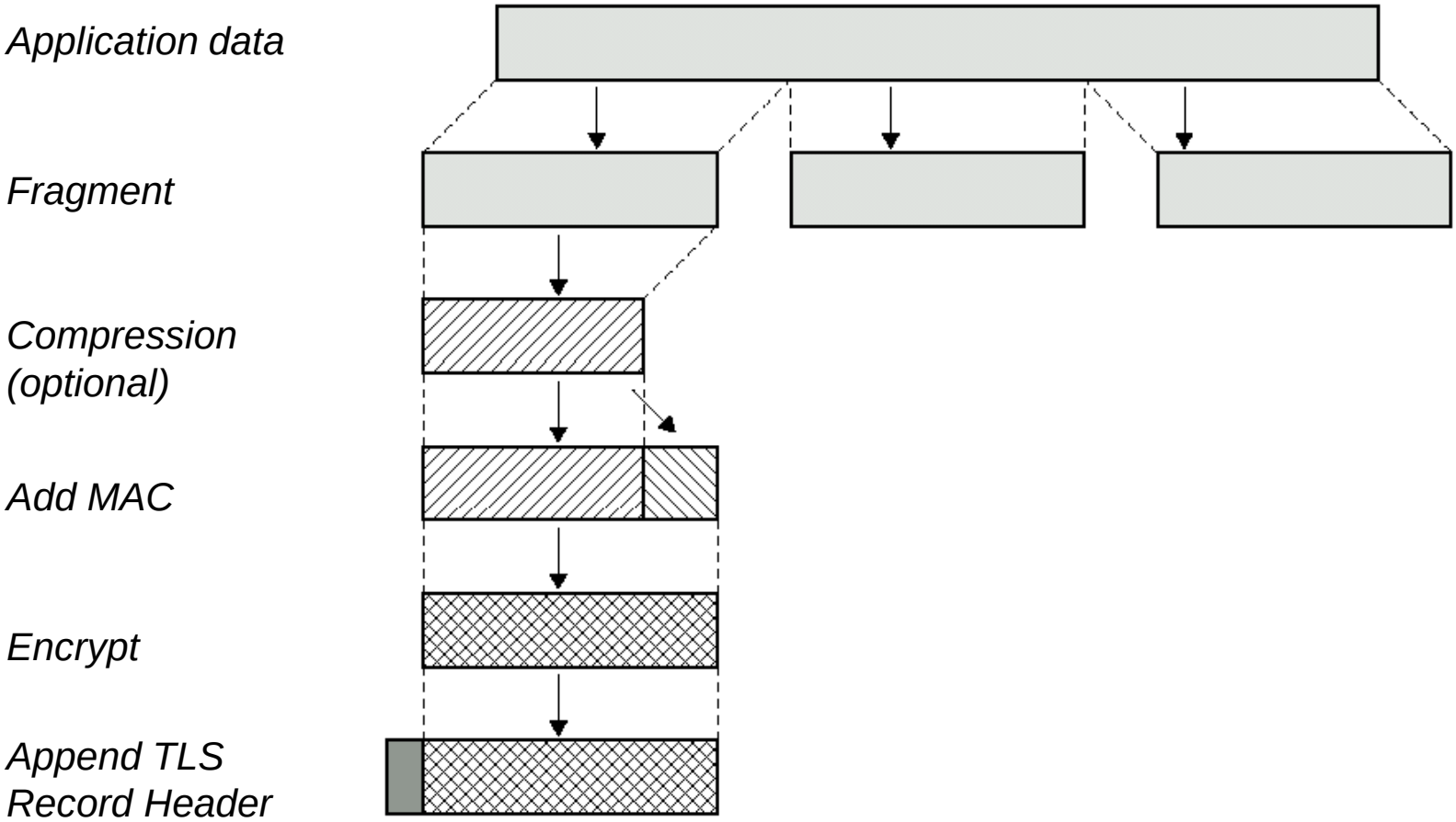


# TLS Record Protocol

- TLS Record Protocol is at the lowest level, layered on top of some reliable transport protocol (e.g., TCP)
- Fragments the data into manageable blocks, optionally compresses the data, applies a MAC, encrypts, and transmits the result
- Provides connection security:
  - **confidentiality**
    - symmetric cryptography is used for data encryption (e.g., AES, RC4, etc.
    - the keys for this symmetric encryption are generated uniquely for each connection and are based on a secret negotiated by another protocol (such as the TLS Handshake Protocol)
  - **Message integrity and data authentication**
    - message transport includes a message integrity check using a keyed MAC
    - secure hash functions (e.g., SHA-1, etc.) are used for MAC computations
- It is used for encapsulating higher-level TLS protocols
  - **currently, four protocols are considered: the handshake protocol, the alert protocol, the change cipher spec protocol, and the application data protocol**

# TLS Record Protocol (cont.)

- TLS-RP operation:

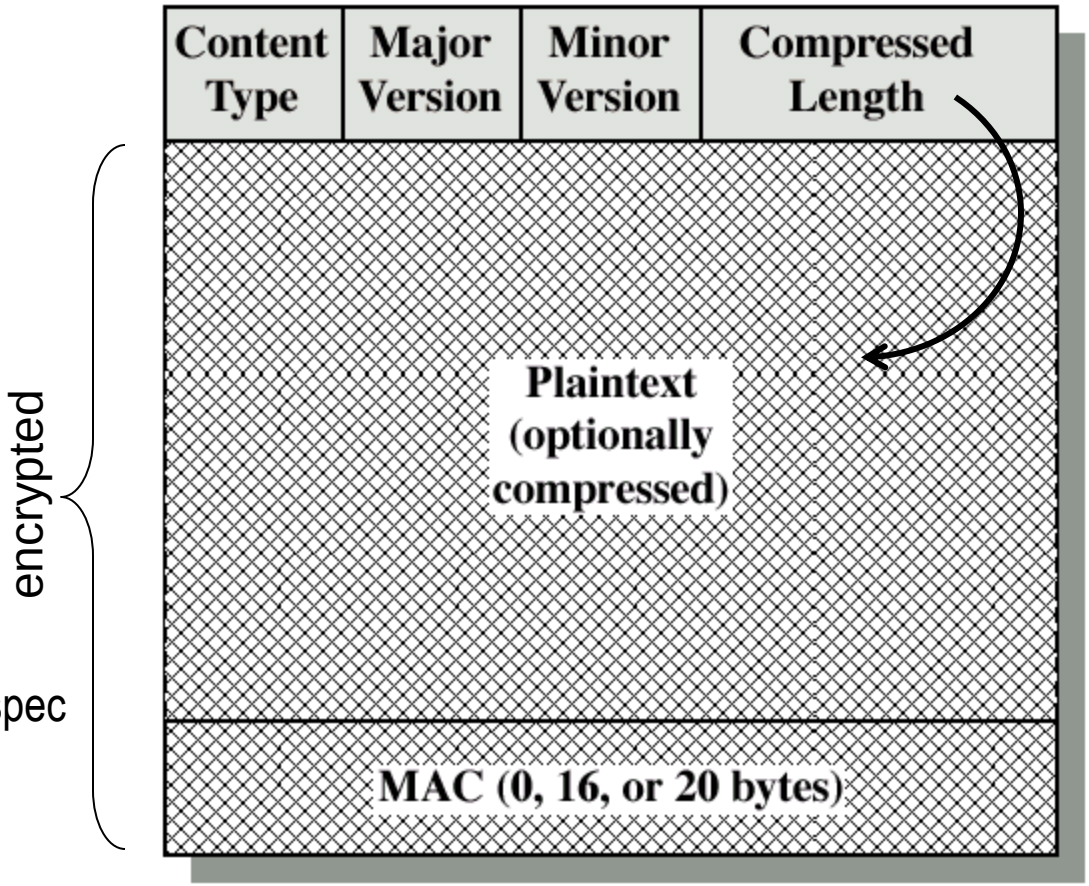


# TLS Record Protocol (cont.)

- TLS-RP format
  - **Major Version=3**
  - **Minor Version:**
    - 0 = SSL3.0
    - 1 = TLS1.0
    - 2 = TLS1.1
    - 3 = TLS1.2

Note: In TLS 1.3, version 0x0304 is included in the *supported_versions* extesion in ClientHello/ServerHello messages

  - **Content Type:**
    - 20 = change_cipher_spec
    - 21 = alert
    - 22 = handshake
    - 23 = application_data



# TLS Handshake Protocol

- TLS Handshake Protocol allows the server and client to authenticate each other and to negotiate an encryption algorithm and cryptographic keys before the application protocol transmits or receives its first byte of data
- The TLS Handshake Protocol provides:
  - **authentication of the the peer's identity using asymmetric cryptography (e.g., RSA, DSA, etc.)**
  - **secure negotiation of a shared secret**
- TLS-RP Content-type 22

# TLS 1.3 key exchanges

- TLS 1.3 supports three basic key exchange modes:
  - **DHE and ECDHE**
    - Diffie-Hellman over either finite fields or elliptic curves is used for secret key establishment
    - the DH exchange (actually the entire handshake) is signed using the server (and optionally the client) certificate's private key
      - SIGMA-like authenticated key exchange
        - » client and server exchange ephemeral DH shares
        - » server signs handshake messages exchanged up to that point (authentication)
        - » both derive shared secrets
      - provides forward secrecy, unilateral or mutual authentication, AKE
  - **PSK alone**
    - Key exchange based on a symmetric Pre-Shared Key
    - the PSK is used as input for the key derivation function
  - **PSK with (EC)DHE**
    - DH exchange is MAC-authenticated using a key derived from PSK

# TLS 1.3 key exchanges (cont.)

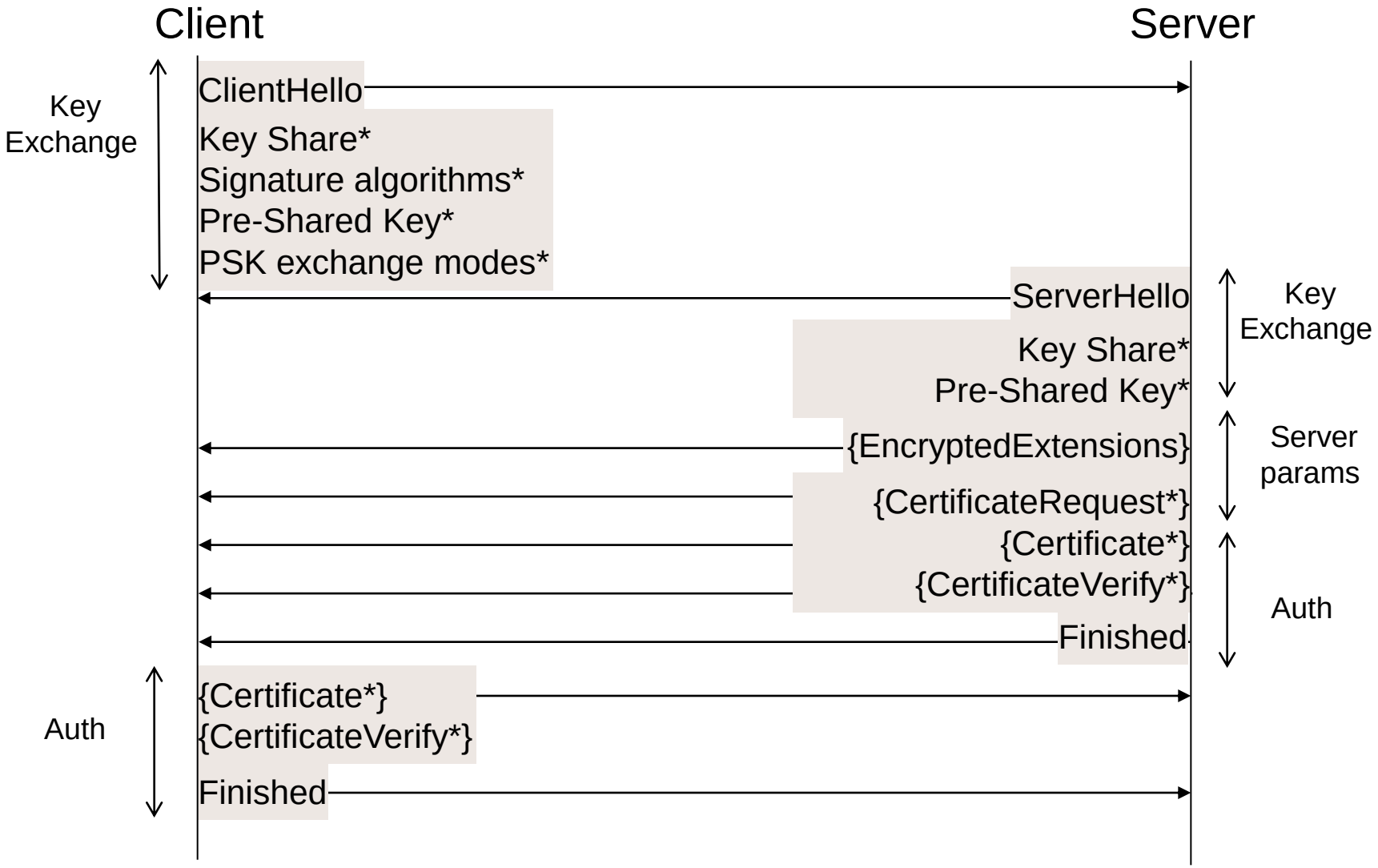
- PSK modes can be used when either:
  - **a PSK is setup within the client and server by a static configuration or by an other dynamic key exchange protocol different from TLS**
    - e.g. useful in case of constrained devices
  - **as result of a TLS handshake from a previous session (session resumption)**
    - used for performance optimization

# Legacy cert-based key exchanges

- Originally TLS supported other cert-based key exchanges:
  - **DHE-RSA and ECDHE-RSA**
    - DH exchange signed using RSA keys
    - forward secrecy
  - **DHE_DSS**
    - DH exchange signed using DSS keys
    - forward secrecy
  - **RSA**
    - key exchanged encrypted by the client with the RSA key of the server
  - **DH_RSA, ECDH-RSA, DH_DSS, and ECDH-ECDSA**
    - use static DH cert (signed with RSA/DSS)

- Removed in TLS 1.3

# TLS Handshake Protocol




* optional messages/extensions
{} protected

# TLS Handshake Protocol (cont.)

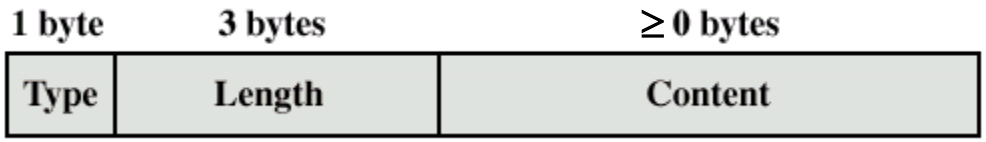
- Key Exchange:
  - **the client sends the ClientHello message**
    - which contains a random nonce, offered protocol versions, list of symmetric cipher/HKDF hash pairs, either a set of Diffie-Hellman key shares (key_share extension), a set of pre-shared key labels (pre_shared_key extension), potentially additional extensions
  - **the server processes the ClientHello and determines the appropriate cryptographic parameters for the connection, then responds with ServerHello message**
    - indicates the negotiated connection parameters
    - the combination of the ClientHello and the ServerHello determines the shared keys
    - if (EC)DHE key establishment is used, the ServerHello contains a key_share extension
      - with the server's ephemeral Diffie-Hellman share
    - if PSK key establishment is used, the ServerHello contains a pre_shared_ke" extension
      - indicating which of the client's offered PSKs was selected

# TLS Handshake Protocol (cont.)

- Certificate-based authentication:
  - **the server sends:**
    - Certificate:
      - the certificate of the endpoint and any per-certificate extensions
      - if raw public keys or the cached information extension are in use, this message will contain other value corresponding to the server's long-term key
    - CertificateVerify:
      - a signature over the entire handshake using the private key
    - Finished:
      - a MAC over the entire handshake
      - it provides key confirmation, binds the endpoint's identity to the exchanged keys, and in PSK mode also authenticates the handshake
  - **the client responds with its Authentication messages, namely Certificate and CertificateVerify (if requested), and Finished**

# TLS Handshake Protocol (cont.)

- TLS Handshake Protocol message format:




- Main TLS 1.3 Handshake Protocol message types:
  - **client_hello(1)**
  - **server_hello(2)**
  - **encrypted_extensions(8)**
  - **certificate(11)**
  - **certificate_request(13)**
  - **certificate_verify(15)**
  - **finished(20)**

# Generating secretes

- Key exchange methods establish a “Pre Master Secret” (PMSK) between the client and server
- For all key exchange methods, the same algorithm is used to convert the PMSK into the “Master Secret” (MSK) [48B]
  - **the PMSK should be deleted from memory once the MSK has been computed**

MSK = PRF(PMSK, "master secret", ClientHello.random || ServerHello.random)

  - **PRF:**
    - PRF(secret, label, data) = P_hash(secret, label || data)
    - P_hash(secret, data) is a data expansion function defined for expand a secret and seed into an arbitrary quantity of output:
      - P_hash(secret, data) = $H_1$ || $H_2$ || $H_3$ || ..
        - » $H_i$ = HMAC_hash(secret, $A_i$ || data)
        - » $A_0$ = data
        - » $A_i$ = HMAC_hash(secret, $A_{i-1}$)
    - P_hash can be iterated as many times as necessary to produce the required quantity of data

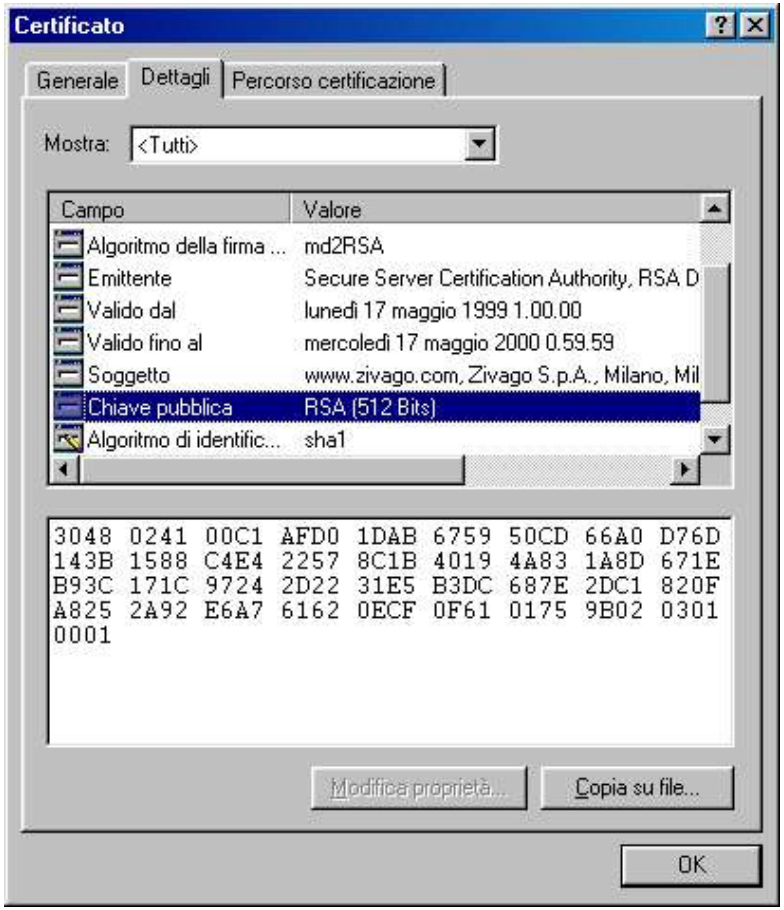
# Generating secretes (cont.)

- MSK is used to generates secrete material such as
  - **a client write MAC secret,**
  - **a server write MAC secret,**
  - **a client write key,**
  - **a server write key,**
  - **a client write IV, and**
  - **a server write IV**
- MSK is hashed into a sequence of secure bytes

  key_block = PRF(MSK, "key expansion", server_random || client_random)
- assigned to the above secret material
  - **MAC secrets, encryption keys, IVs...**

# Web Browser Support

- Some Root/CA certificates pre-installed
  - **Firefox Opera and IE have different lists**
  - **CA's public key is used to verify the signatures on issued certificates**
  - **Browsers can accept unverifiable certificates or alert the user**
- Users can install additional certificates
  - **Additional Root/CA certificates**
    - Security vulnerability
  - **Client certificates**



# Datagram Transport Layer Security (DTLS)

- TLS cannot be directly used with UDP, since packets may be lost or reordered (unreliability)
  - **TLS does not allow independent decryption of individual records**
    - because the integrity check depends on the sequence number, if record N is not received, then the integrity check on record N+1 will be based on the wrong sequence number and thus will fail
    - prior to TLS 1.1, there was no explicit IV and so decryption would also fail
  - **TLS handshake layer assumes that handshake messages are delivered reliably**
    - breaks if those messages are lost
- Datagram Transport Layer Security (DTLS) Version 1.2
  - **TLS over datagram transport (UDP)**
  - **DTLS makes only the minimal changes to TLS required to fix the problem**

# DTLS (cont.)

- Loss-Insensitive Messaging
  - **In TLS Record Layer records are not independent**
    - Cryptographic context of stream cipher key stream is retained between records
    - Anti-replay and message reordering protection are provided by a MAC that includes a sequence number, but the sequence numbers are implicit in the records
  - **DTLS solves the first problem by banning stream ciphers**
  - **DTLS solves the second problem by adding explicit sequence numbers**

# DTLS (cont.)

- Providing Reliability for Handshake
  - **the TLS handshake is a lockstep cryptographic handshake**
    - messages must be transmitted and received in a defined order; any other order is an error
      - this is incompatible with reordering and message loss
  - **DTLS uses a simple retransmission timer to handle packet loss**
  - **in DTLS, each handshake message is assigned a specific sequence number within that handshake**
    - when a peer receives a handshake message, it can quickly determine whether that message is the next message it expects
- TLS handshake messages are potentially larger than any given datagram, thus creating the problem of IP fragmentation
  - **In order to compensate for this limitation, each DTLS handshake message may be fragmented over several DTLS records, each of which is intended to fit in a single IP datagram**
    - each DTLS handshake message contains both a fragment offset and a fragment length